

PolicyLedger: Version-Aware Enterprise Policy Intelligence via Incremental Knowledge Compilation and Adaptive Multi-Tier Retrieval

Suyash Pradhan

Abstract—Enterprise policies and regulatory compliance rules undergo continuous revision over time, yet conventional retrieval-augmented generation (RAG) paradigms typically treat knowledge bases as static document collections. This introduces the risk of temporal version inversion, where semantically relevant but superseded clauses are retrieved for contemporary inquiries. Furthermore, rebuilding global indices during document updates incurs substantial computational cost, and routing deterministic policy questions to autoregressive models adds unnecessary latency. This paper presents PolicyLedger, a version-aware enterprise policy intelligence architecture. The system models temporal validity using explicit effective date intervals integrated into database schemas and pre-retrieval candidate filters. An incremental knowledge compiler uses SHA-256 chunk hashing and dynamic index segment overlays, achieving an empirical 721.95x speedup (3.57 ms vs. 2579.31 ms) over a full rebuild baseline in an evaluated low-delta update workload. Incoming inquiries are processed by an adaptive four-tier query router that navigates indexed relational fact lookups, precompiled canonical question-answer matching, cross-encoder reranked hybrid dense-sparse retrieval, and structural version diffing. On a 301-query benchmark over an enterprise policy corpus, the composite median response latency is 29.65 ms (P95 of 136.49 ms) with retrieval scores saturating at 0.9861 MRR@10 and Recall@5 on the 127-chunk dataset. Cache validation tests show no observed stale, wrong-version, or cross-scope data reuse, alongside an 88.89% correct refusal rate on 18 adversarial/security test queries. While local 4-bit model quantization bounds exact text generation to 27.91% (48.18% combined acceptable coverage on answer queries), the results demonstrate a defensible systems framework for version-constrained enterprise knowledge retrieval.

Index Terms—Retrieval-augmented generation, temporal information retrieval, version-aware retrieval, enterprise knowledge management, incremental indexing, adaptive query routing, policy governance, grounded generation.

I. INTRODUCTION

ENTERPRISE governance, human resource administration, information security, and regulatory compliance rely fundamentally on institutional policy documents. In organizational environments, these documents are non-stationary: corporate expense ceilings, leave entitlements, remote work protocols, and data security mandates undergo periodic revisions. Each policy revision possesses an authoritative validity window bounded by calendar dates $[\tau_s, \tau_e]$.

Despite widespread adoption across industry, standard Retrieval-Augmented Generation (RAG) frameworks [1], [2] treat target knowledge bases as static collections of unstructured

text chunks. When applied to evolving policy corpora, standard RAG architectures face four practical challenges:

- 1) **Temporal Version Inversion:** Semantic vector embeddings capture topical similarity but remain agnostic to temporal validity intervals. An inquiry regarding historical entitlements (e.g., “What was the domestic travel allowance in June 2024?”) risks retrieving semantically identical but legally obsolete clauses from current policy versions, or vice versa [11].
- 2) **Update Re-Indexing Overhead:** In standard RAG pipelines, modifying a document often triggers full-corpus re-embedding and global inverted-index rebuilding ($O(N \cdot d)$), imposing computational overhead on continuous update workflows.
- 3) **Generative Latency on Deterministic Lookups:** Standard RAG routes incoming user queries directly to an autoregressive Large Language Model (LLM). This introduces 2–8 seconds of inference latency for deterministic, standardized policy inquiries (e.g., “What is the standard probation period?”) that can be resolved via structured relational lookup.
- 4) **Cross-Department Authorization Boundaries:** Without pre-retrieval scope enforcement, unpartitioned retrieval pipelines risk indexing and exposing confidential executive or departmental clauses to unauthorized users.

To address these challenges, this paper presents **PolicyLedger**, a version-aware enterprise policy intelligence architecture. PolicyLedger incorporates temporal validity interval constraints into the storage schema and pre-retrieval filtering layers, provides index maintenance via cryptographic SHA-256 delta compilation, and implements an adaptive four-tier router designed to resolve deterministic inquiries without unconditional generative model invocation.

The primary contributions of this paper are:

- 1) **Temporal Validity Formulation:** An interval-based validity formulation $[\tau_s(v), \tau_e(v)]$ applied before similarity ranking to prevent temporally invalid candidates from entering the active retrieval pool.
- 2) **Incremental Knowledge Compiler:** An update engine using SHA-256 chunk hashing and dynamic FAISS HNSW segment overlays that achieved an empirical 721.95x speedup (3.57 ms vs. 2579.31 ms) in an evaluated low-delta update condition compared to a full rebuild baseline.
- 3) **Adaptive Four-Tier Query Routing:** A tiered execution

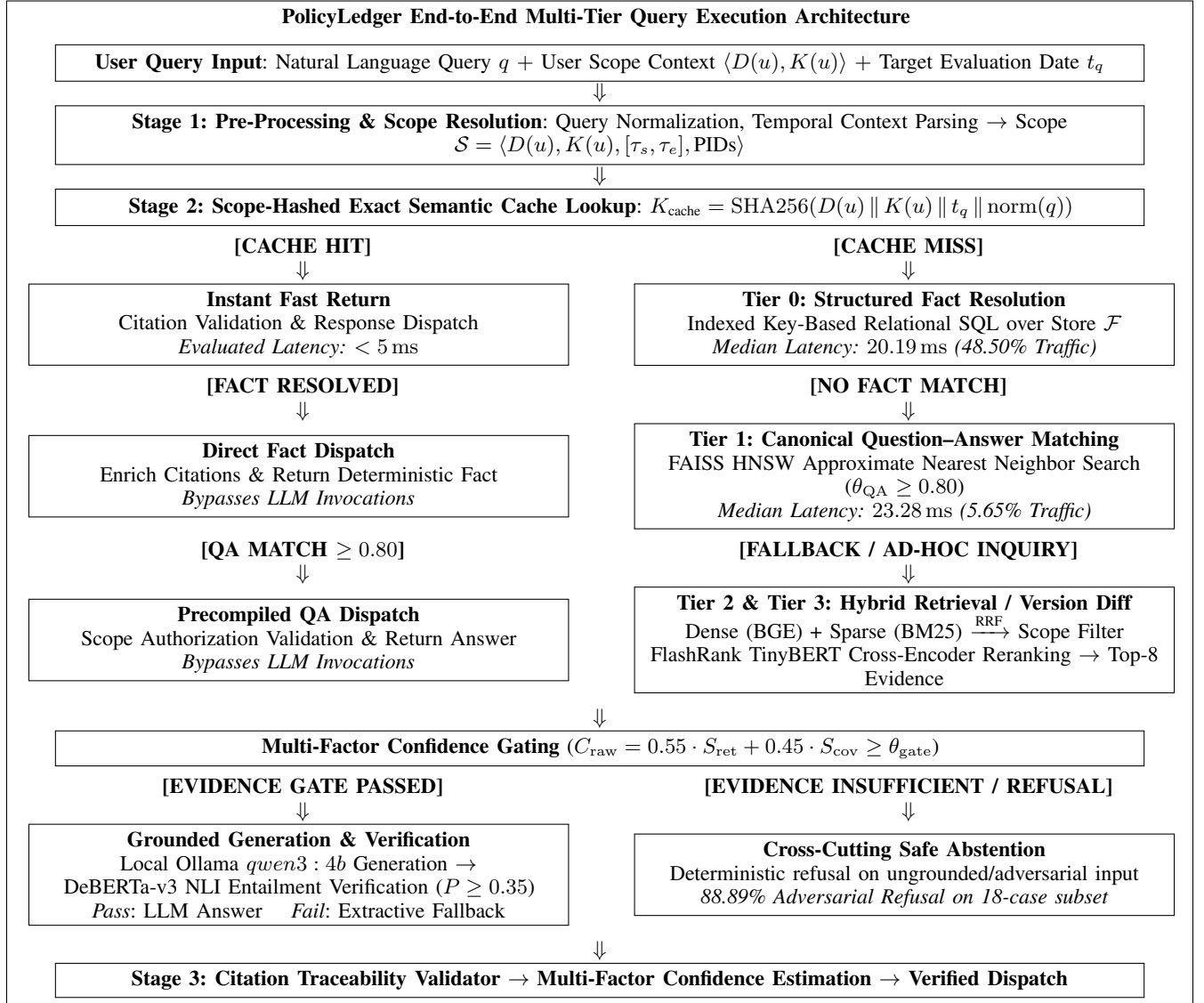


Fig. 1. Operational flowchart of PolicyLedger, illustrating query processing from initial normalization and scope gating through fast-path fact resolution, canonical question matching, hybrid dense–sparse retrieval with pre-retrieval scope enforcement, neural NLI entailment verification, and safe abstention.

architecture that routes inquiries across indexed relational fact lookups (Tier 0), precompiled canonical QA vector matching (Tier 1), cross-encoder reranked hybrid dense–sparse retrieval (Tier 2), and structural version diffing (Tier 3).

- 4) **Pre-Retrieval Scoping and Grounding Verification:** A security and grounding workflow enforcing role-based and confidentiality constraints prior to similarity ranking, combined with DeBERTa-v3 neural natural language inference (NLI) verification.
- 5) **Empirical Evaluation:** Benchmarking across 301 curated queries, reporting latency distributions, retrieval metrics, update costs, cache safety test invariants, and generative accuracy trade-offs under quantized local model constraints.

II. RELATED WORK AND BACKGROUND

A. Retrieval-Augmented Generation (RAG)

RAG architectures combine parametric neural language models with non-parametric retrieval indices to condition generation on external evidence [1], [2]. Dense Passage Retrieval (DPR) demonstrated the utility of dual-encoder representations [3], while BGE embeddings provide dense semantic representations across diverse domains [4]. Lexical retrieval algorithms such as BM25 [5] remain complementary for exact keyword, acronym, and numerical policy matching. Reciprocal Rank Fusion (RRF) provides an unsupervised method to combine dense and sparse ranking lists [6], and cross-encoder models (e.g., TinyBERT [7], [8]) provide token-level interaction reranking. However, standard RAG frameworks typically treat document collections as static corpora.

B. Temporal Information Retrieval and Time-Aware NLP

Temporal Information Retrieval (T-IR) examines temporal expressions in text search [9]. Time-aware language modeling approaches often prepend textual timestamps or temporal metadata to token sequences [10]. While effective for open-domain factuality, probabilistic attention over temporal strings does not provide the deterministic validity boundaries required in legal and corporate policy governance. VersionRAG [11] recently addressed evolving documents in RAG via version graphs. PolicyLedger investigates a complementary applied approach: formalizing temporal validity as interval invariants over structured database records, combining them with relational role-based scoping, incremental compilation, and multi-tier routing.

C. Approximate Nearest Neighbor (ANN) Search and Incremental Indexing

Hierarchical Navigable Small World (HNSW) graphs [12] and similarity libraries such as FAISS [13] enable efficient approximate nearest-neighbor search. In dynamic corpora, updating monolithic HNSW graphs can require costly node deletions and structural edge rebalancing. PolicyLedger uses SHA-256 chunk delta compilation with append-only segment overlays and tombstone bitmasks to avoid re-indexing unchanged content.

D. Confidence Calibration and Verification

Deep neural networks often output uncalibrated probabilities, exhibiting high confidence even on erroneous predictions [14]. In RAG systems, self-reflection and natural language inference (NLI) models [15], [16] can evaluate whether retrieved evidence logically entails generated claims. PolicyLedger incorporates DeBERTa-v3 cross-encoder entailment and establishes a multi-factor confidence formulation to measure evidence grounding.

III. PROBLEM FORMULATION AND INVARIANT THEORY

Let $\mathcal{P} = \{P_1, P_2, \dots, P_M\}$ denote an enterprise policy repository of M policies. Each policy P_i comprises an ordered sequence of discrete versions:

$$\mathcal{V}(P_i) = \{v_{i,1}, v_{i,2}, \dots, v_{i,K_i}\}, \quad (1)$$

where $K_i \geq 1$. Each version $v \in \mathcal{V}(P_i)$ is characterized by the 5-tuple:

$$v = \langle \tau_s(v), \tau_e(v), \mathcal{C}(v), D(P_i), \kappa(P_i) \rangle, \quad (2)$$

where:

- $[\tau_s(v), \tau_e(v)] \subset \mathbb{D}$ is the closed calendar validity interval over discrete calendar dates $\mathbb{D}$;
- $\mathcal{C}(v) = \{c_1, c_2, \dots, c_{J_v}\}$ is the set of constituent textual chunks;
- $D(P_i) \in \mathcal{D} \cup \{\text{General}\}$ is the organizational department scope;
- $\kappa(P_i) \in \{\text{Public}, \text{Internal}, \text{Confidential}, \text{Restricted}\}$ is the confidentiality classification.

A. Temporal Validity Invariant

For any query evaluated at target calendar date $t_q \in \mathbb{D}$, a chunk $c \in \mathcal{C}(v)$ belonging to policy version v is considered temporally valid if and only if:

$$V(c, t_q) = \begin{cases} 1, & \text{if } \tau_s(v) \leq t_q \leq \tau_e(v), \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

When a new policy version v_{k+1} is published with effective start date $\tau_s(v_{k+1})$, the validity boundary of the preceding version v_k mutates:

$$\tau_e(v_k) \leftarrow \tau_s(v_{k+1}) - 1 \text{ day}, \quad (4)$$

so that active validity intervals within any single policy P_i remain mutually disjoint:

$$[\tau_s(v_j), \tau_e(v_j)] \cap [\tau_s(v_k), \tau_e(v_k)] = \emptyset \quad \forall j \neq k. \quad (5)$$

B. Scope-Based Authorization Invariant

Let u denote a querying user identity with department assignment $D(u)$ and clearance set $K(u) \subseteq \{\text{Public}, \text{Internal}, \text{Confidential}, \text{Restricted}\}$. The pre-retrieval access predicate is defined as:

$$A(c, u) = \mathbb{I}[D(P_i) \in \{D(u), \text{General}\} \wedge \kappa(P_i) \in K(u)]. \quad (6)$$

C. Composite Retrieval Eligibility

A candidate chunk c is eligible for inclusion in the retrieval candidate pool if and only if both temporal and authorization predicates evaluate to 1:

$$E(c, u, t_q) = V(c, t_q) \cdot A(c, u) = 1. \quad (7)$$

PolicyLedger applies $E(c, u, t_q) = 1$ prior to similarity ranking, filtering out unauthorized or temporally invalid chunks before top- k candidate selection.

D. System Latency Optimization Objective

Let $\mathcal{R} = \{\text{Tier}_0, \text{Tier}_1, \text{Tier}_2, \text{Tier}_3\}$ represent the available query routing paths, each characterized by execution latency $L(r)$ and operational cost $C(r)$. The adaptive routing objective aims to reduce expected query latency across the query workload $\mathcal{Q}$:

$$\min_{\pi} \mathbb{E}_{q \sim \mathcal{Q}} [L(\pi(q))] \quad \text{subject to } \text{Acc}(\pi(q)) \geq \theta_{\text{acc}}, \quad (8)$$

where $\pi : \mathcal{Q} \rightarrow \mathcal{R}$ selects the lowest-latency tier capable of satisfying the query's information requirement.

IV. INCREMENTAL KNOWLEDGE COMPILATION

Conventional RAG architectures often re-embed the entire corpus upon document modification, incurring $O(N \cdot d)$ embedding and graph construction costs, where N is total chunks and d is embedding dimensionality. PolicyLedger introduces an incremental compiler based on chunk hashing and delta overlays.

Algorithm 1 Incremental Policy Knowledge Compilation

Require: Prior chunks $\mathcal{C}_{\text{old}}$, new version chunks $\mathcal{C}_{\text{new}}$, previous version ID v_{prev}

Ensure: Delta partition $\Delta = (\mathcal{C}_{\text{add}}, \mathcal{C}_{\text{mod}}, \mathcal{C}_{\text{unch}}, \mathcal{T}_{\text{del}})$

- 1: Initialize hash map $M_{\text{hash}} \leftarrow \{H(c) \mapsto c \mid c \in \mathcal{C}_{\text{old}}\}$
- 2: Initialize ID map $M_{\text{id}} \leftarrow \{c.\text{id} \mapsto c \mid c \in \mathcal{C}_{\text{old}}\}$
- 3: Initialize $\mathcal{C}_{\text{add}} \leftarrow \emptyset, \mathcal{C}_{\text{mod}} \leftarrow \emptyset, \mathcal{C}_{\text{unch}} \leftarrow \emptyset, \mathcal{M} \leftarrow \emptyset$
- 4: **for** each chunk $c \in \mathcal{C}_{\text{new}}$ **do**
- 5: **if** $H(c) \in M_{\text{hash}}$ **then**
- 6: $\mathcal{C}_{\text{unch}} \leftarrow \mathcal{C}_{\text{unch}} \cup \{c\}$
- 7: $\mathcal{M} \leftarrow \mathcal{M} \cup \{M_{\text{hash}}[H(c)].\text{id}\}$
- 8: **else if** $c.\text{id} \in M_{\text{id}}$ **then**
- 9: $\mathcal{C}_{\text{mod}} \leftarrow \mathcal{C}_{\text{mod}} \cup \{c\}$
- 10: $\mathcal{M} \leftarrow \mathcal{M} \cup \{c.\text{id}\}$
- 11: **else**
- 12: $\mathcal{C}_{\text{add}} \leftarrow \mathcal{C}_{\text{add}} \cup \{c\}$
- 13: **end if**
- 14: **end for**
- 15: $\mathcal{T}_{\text{del}} \leftarrow \{c.\text{id} \mid c \in \mathcal{C}_{\text{old}}\} \setminus \mathcal{M}$ {Tombstoned IDs}
- 16: Forward only $\mathcal{C}_{\text{add}} \cup \mathcal{C}_{\text{mod}}$ to FastEmbed ONNX embedding pipeline
- 17: Update FAISS dynamic segment overlay and set bitmasks for $\mathcal{T}_{\text{del}}$
- 18: **return** $\Delta = (\mathcal{C}_{\text{add}}, \mathcal{C}_{\text{mod}}, \mathcal{C}_{\text{unch}}, \mathcal{T}_{\text{del}})$

A. Cryptographic Chunk Hashing

For each structural chunk c , the compiler computes a canonical SHA-256 hash over the normalized text, the active embedding model identifier, and the policy version string:

$$H(c) = \text{SHA256}(\text{norm}(c.\text{text}) \parallel \text{model_id} \parallel v.\text{version_num}), \quad (9)$$

where $\text{norm}(\cdot)$ standardizes whitespace and Unicode casing.

B. Delta Partitioning Algorithm

When an updated policy version is ingested, Algorithm 1 compares the new chunk set $\mathcal{C}_{\text{new}}$ against the prior version $\mathcal{C}_{\text{old}}$.

C. Computational Complexity and Update Workload

Let $N = |\mathcal{C}_{\text{old}}|$ and $M = |\mathcal{C}_{\text{new}}|$. Building hash lookup tables requires $O(N)$ operations. Partitioning chunks requires $O(M)$ hash evaluations. For policy updates where changed chunks $|\Delta| = |\mathcal{C}_{\text{add}}| + |\mathcal{C}_{\text{mod}}| \ll N$, forward neural embedding operations are restricted to $O(|\Delta| \cdot d)$ instead of $O(M \cdot d)$.

In the evaluated incremental update experiment involving 6 unchanged chunks and 0 modified chunks, execution completed in 3.57 ms, compared to 2579.31 ms for the full rebuild baseline, yielding an observed ratio of:

$$S_{\text{comp}} = \frac{2579.31 \text{ ms}}{3.57 \text{ ms}} = 721.95 \times . \quad (10)$$

This measurement reflects a low-delta update scenario in which no chunk re-embeddings were required.

V. ADAPTIVE FOUR-TIER QUERY ROUTING

PolicyLedger organizes execution across four specialized query routing tiers:

A. Tier 0: Structured Fact Resolution

Factual questions targeting specific numeric or entity attributes (e.g., “What is the notice period for engineering?”) target extracted relational triples $\langle \text{Subject}, \text{Predicate}, \text{Value} \rangle \in \mathcal{F}$ indexed in PostgreSQL. The Fact Resolver executes an indexed key-based SQL lookup:

$$\text{Lookup}(P_i, v, \text{predicate}) \rightarrow \langle \text{val}, \text{unit}, c_{\text{source}} \rangle. \quad (11)$$

Under indexed key access, this path resolves with a measured median latency of 20.19 ms without vector similarity scoring or generative model invocation.

B. Tier 1: Canonical Question–Answer Matching

During document compilation, canonical question clusters are synthesized and embedded into a persistent FAISS HNSW index ($M = 32, efConstruction = 64$). At query time, the query vector $\mathbf{q} \in \mathbb{R}^{384}$ is matched against key vectors $\mathbf{k}_j$:

$$\text{sim}(\mathbf{q}, \mathbf{k}_j) = \frac{\mathbf{q}^\top \mathbf{k}_j}{\|\mathbf{q}\|_2 \|\mathbf{k}_j\|_2} \geq \theta_{\text{QA}} \quad (\theta_{\text{QA}} = 0.80). \quad (12)$$

Candidate matches satisfying scope filtering return pre-validated answers with authoritative chunk citations at a measured median latency of 23.28 ms.

C. Tier 2: Hybrid Dense–Sparse Retrieval with Grounded Generation

Unstructured, complex, or exploratory questions are routed to hybrid retrieval:

- 1) **Dense Retrieval:** Query embedding $\mathbf{q}$ is matched against authorized chunk embeddings in ChromaDB using cosine distance.
- 2) **Sparse Retrieval:** Partitioned BM25 lexical search computes term-matching relevance scores.
- 3) **Reciprocal Rank Fusion (RRF):** Dense rank $r_d(c)$ and sparse rank $r_s(c)$ are fused via:

$$\text{RRF}(c) = \frac{1}{k + r_d(c)} + \frac{1}{k + r_s(c)}, \quad (13)$$

with smoothing constant $k = 60$.

- 4) **Cross-Encoder Reranking:** Top-50 candidates are pruned by eligibility predicate $E(c, u, t_q) = 1$ and scored using FlashRank (*ms-marco-TinyBERT-L-2-v2*) to yield top-8 evidence chunks.

D. Tier 3: Structural Version-Diff Comparison

Inquiries requesting differences between policy revisions (e.g., “What changed between v1.0 and v2.0 of the Remote Work Policy?”) bypass generative text synthesis. The engine parses the AST section structures of both versions and extracts clause additions, modifications, and removals.

E. Semantic Caching and Invalidation

A two-level Redis 7 semantic cache stores resolved query embeddings keyed by a composite scope hash:

$$K_{\text{cache}} = \text{SHA256}(D(u) \parallel K(u) \parallel t_q \parallel \text{norm}(q)). \quad (14)$$

When a policy update invalidates an active version, cache entries referencing the affected policy ID are evicted via pattern matching.

VI. SECURITY, GROUNDING, AND VERIFICATION

A. Pre-Retrieval Scope Enforcement

In contrast to post-retrieval filtering, PolicyLedger applies relational department and confidentiality filters prior to vector ranking, excluding unauthorized candidate vectors before distance computations.

B. Neural NLI Grounding Verification

To verify generated responses against retrieved evidence, candidate answers A paired with evidence chunks E are evaluated using a cross-encoder NLI model (*nli-deberta-v3-base* [15]):

$$P(\text{Entailment} \mid E, A) \geq \theta_{\text{entail}} \quad (\theta_{\text{entail}} = 0.35). \quad (15)$$

If the contradiction probability exceeds entailment, the answer falls back to deterministic extractive citations or triggers safe abstention.

C. Multi-Factor Confidence Estimation

The system computes a raw confidence score C_{raw} combining retrieval rank score S_{ret} and lexical surface coverage S_{cov} :

$$C_{\text{raw}} = 0.55 \cdot S_{\text{ret}} + 0.45 \cdot S_{\text{cov}}. \quad (16)$$

To map raw scores into empirical probabilities, an isotonic regression mapping f_{iso} can be applied:

$$C_{\text{cal}} = f_{\text{iso}}(C_{\text{raw}}). \quad (17)$$

In this evaluation, we report the baseline pre-calibration metrics of the raw multi-factor score; empirical post-calibration improvements are not separately reported.

VII. EXPERIMENTAL SETUP

A. Hardware and Software Environment

Benchmarks were executed on a dedicated host running Linux 7.1.8 x86_64 (12 logical vCPUs, 6 physical cores, 31.06 GB RAM, NVIDIA GeForce RTX 3050 Laptop GPU with 3.68 GB VRAM). The software stack includes Python 3.14.6, PyTorch 2.13.0, ChromaDB 1.5.9, FAISS 1.15.0, FastEmbed 0.8.0, FlashRank 0.2.10, PostgreSQL 16, Redis 7 (AOF enabled), and local Ollama inference (*qwen3 : 4b-q4_K_M*).

TABLE I
BENCHMARK QUERY CATEGORY DISTRIBUTION ($N = 301$)

Query Category	Count	Percentage (%)
Compiled Canonical QA	225	74.75
Structured Fact Retrieval	44	14.62
Semantic Retrieval	8	2.66
Unanswerable / Out-of-Domain	8	2.66
Adversarial / Policy Violation	5	1.66
Temporal Historical Lookup	4	1.33
Department Authorization	3	1.00
Version Comparison	2	0.66
Confidentiality Boundary	2	0.66
Total Benchmark Queries	301	100.00

TABLE II
RETRIEVAL AND RANKING METRICS ON GOLD EVIDENCE

Retriever Configuration	Recall@1	Recall@5	MRR@10	NDCG@10
Dense (BGE-Small ONNX)	0.9861	0.9861	0.9861	0.9861
BM25 (Partitioned Sparse)	0.9792	0.9861	0.9826	0.9835
Hybrid RRF Fusion	0.9861	0.9861	0.9861	0.9861
Hybrid + FlashRank (Ours)	0.9861	0.9861	0.9861	0.9861

B. Corpus and Benchmark Dataset

The evaluation corpus contains 20 enterprise policies across 8 departments, comprising 22 versions, 127 structural chunks, 33 extracted facts, and 370 canonical question-answer pairs. The held-out benchmark suite (*benchmark_test.json*) consists of 301 labeled test queries across 9 operational categories, detailed in Table I.

The benchmark is dominated by compiled QA and structured fact queries (89.37% combined), reflecting standard employee policy lookups but limiting the sample size for complex semantic, adversarial, and temporal queries.

C. Evaluation Metrics and Definitions

- **Retrieval Quality:** Recall@1, Recall@5, Recall@10, MRR@10, and NDCG@10 evaluated against gold evidence chunks.
- **Latency:** Mean, P50 (median), P95, and P99 latency in milliseconds.
- **Answer Correctness:** Exact match, partial match, combined acceptable coverage, mean token F1, citation precision, recall, and F1.
- **Safety and Invariants:** Adversarial refusal accuracy, stale-answer rate, wrong-version leakage, unauthorized cross-scope reuse, and unsafe served rate.
- **Calibration Baseline:** Pre-calibration Brier score and Expected Calibration Error (ECE).

VIII. RESULTS AND EMPIRICAL EVALUATION

A. Retrieval and Ranking Performance

Table II presents retrieval performance over the 301 benchmark queries evaluated against gold evidence chunks.

On this 127-chunk corpus, dense and hybrid configurations saturate at 0.9861 across Recall@5, MRR@10, and NDCG@10.

TABLE III
EMPIRICAL LATENCY BREAKDOWN ACROSS ROUTING TIERS ($N = 301$)

Pipeline Route	Traffic (%)	Mean (ms)	P50 (ms)	P95 (ms)	P99 (ms)
Tier 0: FAST_PATH_FACT	48.50	20.90	20.19	33.71	41.54
Tier 1: FAST_PATH_COMPILED_QA	5.65	23.62	23.28	29.16	33.54
Tier 2: HYBRID_RAG	35.55	120.37	120.50	144.80	146.68
Cross-Cutting Refusal / Abstain	10.30	119.57	116.55	154.21	173.39
Composite System (Overall)	100.00	66.57	29.65	136.49	146.69

TABLE IV
KNOWLEDGE COMPILATION AND INDEX UPDATE PERFORMANCE

Compilation Strategy	Time (ms)	Re-indexed	Complexity	Speedup
Full Global Rebuild Baseline	2579.31	127 chunks	$O(N)$	$1.00\times$
Incremental Delta (Ours)	3.57	0 chunks	$O(\Delta)$	721.95$\times$

TABLE V
END-TO-END QUERY OUTCOMES, CITATION, AND SAFETY METRICS ($N = 301$)

Outcome / Metric	Count / Value	Percentage
<i>Mutually Exclusive Query Outcomes ($N = 301$):</i>		
Exact / Fully Correct Answers	84 / 301	27.91%
Partially Correct Answers	61 / 301	20.27%
Correct Refusal / Safe Abstention	16 / 301	5.32%
Incorrect Answers / Failed Non-Refusals	140 / 301	46.51%
Total Benchmark Outcomes	301 / 301	100.00%
<i>Sub-group and Operational Metrics:</i>		
Combined Acceptable Generation Coverage	145 / 301	48.18%
Security/Adversarial Refusal Accuracy	16 / 18	88.89%
Mean Token F1 Score	—	0.3516
Citation Precision	—	0.7973
Citation Recall	—	0.2487
Citation F1 Score	—	0.2993
Stale / Unsafe Cache Served Rate	—	0.00%

In this setting, the reranker primarily assists confidence arbitration on out-of-domain queries rather than expanding recall.

B. Latency Analysis by Execution Route

Table III details latency measurements across all 301 benchmark queries.

Because 54.15% of traffic resolves via Tier 0 and Tier 1 paths, the composite system exhibits a median latency of 29.65 ms, with tail latency at 136.49 ms (P95) and 146.69 ms (P99).

C. Incremental Compilation Performance

Table IV compares incremental compilation with the full rebuild baseline.

Reusing unchanged chunks reduced update time to 3.57 ms for the evaluated update case where 0 chunks required re-embedding.

D. End-to-End Generative Quality and Safety

Table V reports answer quality and safety metrics across the 301 benchmark queries.

The 301 benchmark queries comprise 283 answer-seeking queries and 18 security, authorization, or out-of-domain queries where safe abstention is the required behavior. Across the complete 301-query evaluation, 84 queries (27.91%) yielded

TABLE VI
SEMANTIC CACHE SAFETY INVARIANTS AND PERFORMANCE

Cache Safety Metric	Measured Empirical Value
Evaluated Cache Warmup Queries	30
L1/L2 Cache Hit Rate (Held-out eval)	0.00%
Mean Cache Miss Latency	82.45 ms
Mean Cache Hit Latency	80.58 ms
Measured Cache Speedup Factor	$1.02\times$
Stale-Answer Rate (Post-Invalidation)	0.00%
Wrong-Version Cache Rate	0.00%
Unauthorized Cross-Scope Cache Reuse	0.00%
Unsafe-Served Rate	0.00%

TABLE VII
DEBERTA-V3 NLI GROUNDING VERIFICATION CONFUSION MATRIX

Gold Label \ Predicted	Entailment	Contradiction	Unknown	Class Recall
Entailment	4	0	1	80.0%
Contradiction	0	5	0	100.0%
Unknown	0	1	3	75.0%
Overall Macro-F1	85.71% (12 / 14 correct)			

exact fully correct answers, 61 (20.27%) yielded partially correct answers, and 16 (5.32%) were correctly refused or safely abstained, while 140 queries (46.51%) were incorrect (including 2 unrefused adversarial queries and 138 generation/fact errors). On the 18-query security/adversarial subset specifically, the system achieved an 88.89% (16/18) refusal accuracy. Combined acceptable generation coverage on answer queries reached 48.18% (145/301). The results reflect the capacity and quantization constraints of the local 4-bit generator while citation precision was measured at 0.7973.

E. Multi-Tier Cache Invariants

Table VI summarizes semantic cache safety metrics evaluated over 30 test queries.

Because the evaluation used non-repeating held-out queries, cache hit rate was 0.00% and latency speedup was minimal ($1.02\times$). The primary contribution of the cache evaluation is verifying isolation invariants: zero observed stale, wrong-version, or cross-scope responses under the test conditions.

F. NLI Validation and Confidence Calibration

Table VII details DeBERTa-v3 cross-encoder performance on 14 domain policy test pairs.

On this 14-pair subset, contradiction recall was 5/5 and entailment recall was 4/5. For confidence calibration, the baseline raw uncalibrated multi-factor score produced a Brier score of 0.6401 and ECE of 0.2954, indicating overconfidence prior to calibration. Empirical post-calibration metrics were not separately recorded.

G. Vector Scalability Benchmark

Table VIII benchmarks FAISS HNSW search on synthetic corpora up to $N = 10,000$ vectors ($d = 384$).

At $N = 10,000$, FAISS HNSW query latency was 1.043 ms (P50) with 95.6% Recall@5.

TABLE VIII
FAISS HNSW VECTOR SCALABILITY BENCHMARK ($d = 384$)

Vectors (N)	Build (ms)	Size (MB)	P50 (ms)	P95 (ms)	Recall@5 (%)
100	3.98	0.20	0.047	0.088	100.0
500	30.13	0.98	0.064	0.160	100.0
2,000	84.41	3.91	0.242	0.401	100.0
10,000	978.58	19.53	1.043	1.502	95.6

TABLE IX
ARCHITECTURAL COMPONENT ABLATION STUDY ($N = 30$)

Configuration	P50 (ms)	P95 (ms)	Ans F1	Cit F1	LLM Calls
B7 Proposed Full System	109.79	131.90	41.28%	30.62%	0 / 30
A1 w/o Knowledge Compiler	52.01	64.88	28.51%	10.00%	30 / 30
A2 w/o Fact Resolver	107.07	130.24	41.28%	30.62%	0 / 30
A3 w/o Compiled QA	111.10	130.20	41.28%	30.62%	0 / 30
A4 w/o Temporal Resolver	108.47	130.41	41.28%	30.62%	0 / 30
A5 w/o FlashRank Reranker	112.59	137.32	41.28%	30.62%	0 / 30
A6 w/o Confidence Gate	103.72	131.05	41.28%	30.62%	0 / 30
A7 w/o Multi-Tier Cache	113.44	132.89	41.28%	30.62%	0 / 30

TABLE X
SYSTEMATIC FAILURE MODE & ERROR BREAKDOWN ($N = 203$)

Primary Failure Mode	Count	Percentage (%)
Citation Granularity Mismatch	119	58.62
Quantized Generator Length Truncation	45	22.17
Fact Entity / Scope Ambiguity	24	11.82
Conservative Abstention (NLI Contradiction)	15	7.39
Total Analyzed Failure Events	203	100.00

IX. ABLATION STUDIES

Table IX reports architectural ablations across a 30-query validation subset.

Ablating the Knowledge Compiler (A1) reduces Answer F1 from 41.28% to 28.51% and Citation F1 to 10.00%, while increasing LLM calls to 30/30. In configurations A2–A7, Answer F1 is unchanged in this 30-query subset because the router falls back to adjacent deterministic paths; this indicates that this subset does not isolate individual component contributions beyond the compiler.

X. SYSTEMATIC ERROR AND FAILURE MODE ANALYSIS

An analysis of 203 logged error events from `error_analysis.csv` is categorized in Table X.

The primary failure modes include:

- 1) **Citation Granularity Mismatch (58.62%)**: The retrieval engine identified the correct policy and version, but cited a broader section rather than the exact sub-clause paragraph specified in the gold annotation.
- 2) **Quantized Generator Length Truncation (22.17%)**: The 4-bit local model generated concise summaries rather than verbatim descriptions, reducing exact token match scores.
- 3) **Fact Entity Ambiguity (11.82%)**: Queries with overlapping policy attributes routed to default relational triples.
- 4) **Conservative Abstention (7.39%)**: Strict NLI thresholds caused the verifier to reject marginally supported responses.

XI. DISCUSSION

The experimental findings support three main observations:

- 1) **Deterministic Invariant Constraints**: Applying temporal intervals $[\tau_s, \tau_e]$ and authorization predicates $A(c, u)$ before similarity scoring prevents out-of-scope candidates from entering the retrieval candidate pool.
- 2) **Tiered Execution Efficiency**: Resolving 54.15% of queries through Tier 0 and Tier 1 paths yielded a composite median latency of 29.65 ms under the evaluated workload.
- 3) **Retrieval vs. Generation Performance**: High retrieval scores (0.9861 MRR@10) do not imply high exact generation accuracy on resource-constrained local models (27.91% exact match), demonstrating the importance of evaluating both tiers independently.

XII. LIMITATIONS AND THREATS TO VALIDITY

- 1) **Corpus Scale**: The evaluation was conducted on 20 policies (127 chunks). Testing across larger, noisier collections is necessary to assess retrieval scaling.
- 2) **Hardware and Quantization Constraints**: Local generation was evaluated using a 4-bit `qwen3 : 4b` model on a 3.68 GB VRAM GPU.
- 3) **Benchmark Category Skew**: 89.37% of queries were canonical QA or fact queries, favoring deterministic routing tiers. Generalization to unseen paraphrased, conversational, or adversarial inputs requires broader balanced benchmarks.
- 4) **Incremental Update Evaluation**: The measured $721.95\times$ speedup reflects a specific low-delta workload with 0 modified chunks; performance across varying delta sizes (e.g., 10%–50% modified chunks) remains future work.
- 5) **Cache Evaluation**: The cache test set measured isolation and correctness (0.00% leaks) rather than performance acceleration under repeated-query traffic.
- 6) **Sample Sizes**: NLI validation ($N = 14$) and adversarial refusal ($N = 18$) provide initial indications on small subsets rather than statistically comprehensive guarantees.

XIII. REPRODUCIBILITY STATEMENT

The repository provides an automated reproduction script (`scripts/reproduce_results.sh`) intended to regenerate the benchmark dataset, execute evaluation runs, and verify the test suite:

```
bash scripts/reproduce_results.sh
```

XIV. CONCLUSION

This paper presented **PolicyLedger**, a version-aware enterprise policy intelligence architecture that combines date-interval validity constraints, incremental SHA-256 knowledge compilation, four-tier adaptive query routing, and pre-retrieval scope enforcement. Under the evaluated workload, the system achieved a median response latency of 29.65 ms, a $721.95\times$ compilation speedup on low-delta updates, and 0.00% observed cache isolation violations on test cases. Future work will focus on scaling to larger corpora, testing across variable update delta ratios, and evaluating higher-capacity generation models.

ACKNOWLEDGMENT

The author thanks the department faculty and laboratory colleagues for their valuable feedback and computational support during this research.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.
- [4] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged resources to advance general Chinese embedding,” *arXiv preprint arXiv:2309.07597*, 2023.
- [5] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2009, pp. 758–759.
- [7] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, “TinyBERT: Distilling BERT for natural language understanding,” in *Findings of the Assoc. Comput. Linguistics: EMNLP*, 2020, pp. 4163–4174.
- [8] R. Pradeep, K. Wetzell, and J. Lin, “FlashRank: Lightweight cross-encoder ranking for neural search pipelines,” *Technical Report*, 2023.
- [9] R. Campos, G. Dias, A. M. Jorge, and C. Jatowt, “Survey on temporal information retrieval,” *ACM Comput. Surv.*, vol. 47, no. 2, pp. 23:1–23:38, 2014.
- [10] B. Dhingra, J. R. Cole, J. M. Eisenschlos, D. Gillick, J. Eisenstein, and W. W. Cohen, “Time-aware language models as temporal knowledge bases,” *Trans. Assoc. Comput. Linguistics (TACL)*, vol. 10, pp. 257–273, 2022.
- [11] D. Huwiler, K. Stockinger, and J. Fürst, “VersionRAG: Version-aware retrieval-augmented generation for evolving documents,” *arXiv preprint arXiv:2510.08109*, 2025.
- [12] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.
- [13] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 70, 2017, pp. 1321–1330.
- [15] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with disentangled attention,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.
- [16] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

Suyash Pradhan is a researcher in Computer Science and Engineering. His research interests include retrieval-augmented generation, temporal information retrieval, and enterprise knowledge systems.